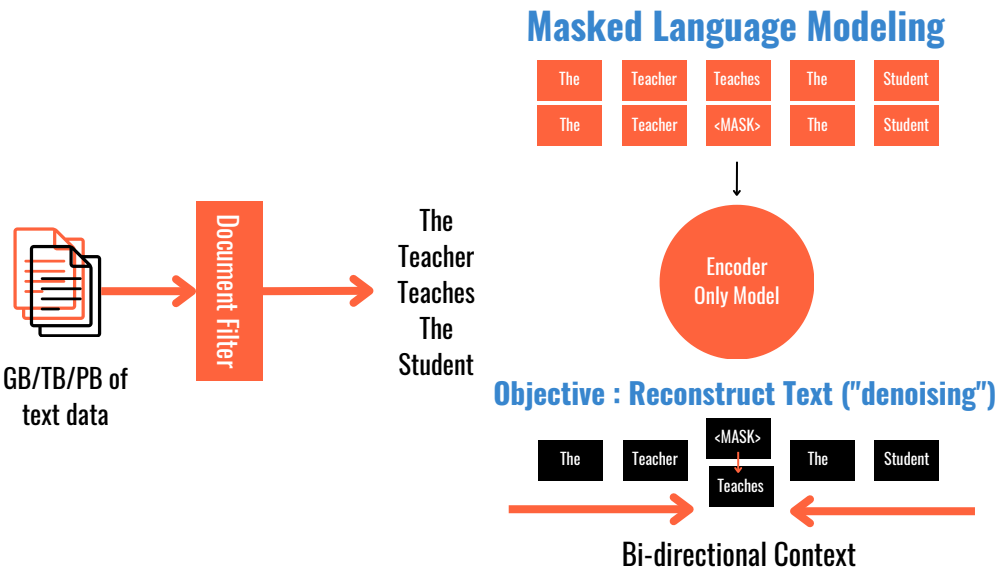
How do you pre-train Large Language Models?

Three LLM Architectures

AutoEncoding Models (Encoder Only)



USE CASES

- Sentiment Analysis
- Named Entity Recognition
- Word Classification

EXAMPLES

- BERT
- ROBERTA

Finetuning Foundation Models vs Pretaining

For most requirements, finetuning an existing LLM will suffice. However, there can be cases when pre-training a new LLM will provide better application especially when the language is highly domain specific e.g Legal, Medical. Pre-training is a resource intensive and costly process.

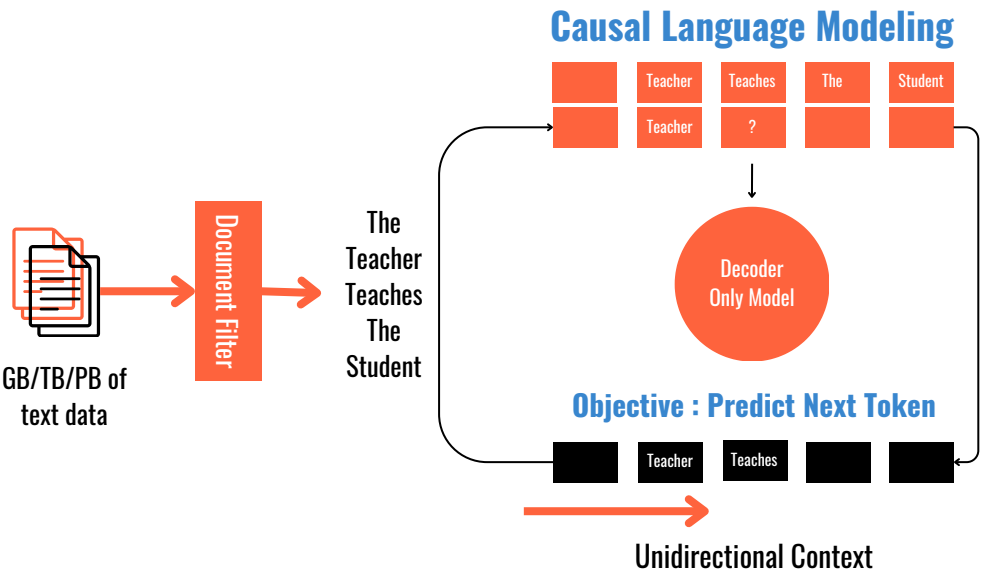
Open Source LLMs

While OpenAI's proprietary GPT-3.5, GPT-4 have gained immense popularity, HuggingFace Model Hub provides access to powerful Open Source LLMs along with documentation and training architecture. Architecture plays a critical role in defining what objective can each LLM be used for.

Training Data and Model Size

LLMs are generally trained on Petabytes of data, mostly from the open internet. The unstructured text requires careful filtering. As a result only 2-3% of data is useful for training.
LLM size is measured in terms of the number of parameters. Larger models have generalised well to a variety of tasks.

AutoRegressive Models (Decoder Only)



USE CASES

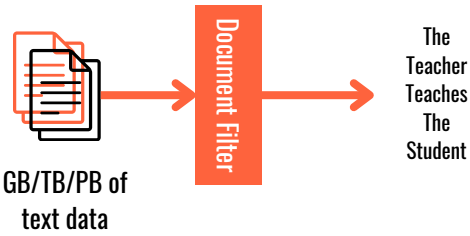
- Text Generation

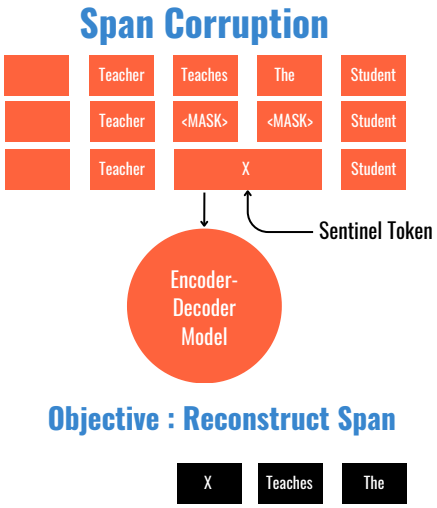
  (Most common architecture now and larger models can perform a variety of tasks)

EXAMPLES

- GPT
- BLOOM

Sequence-to-Sequence Models (Encoder-Decoder)





USE CASES

- Translation
- Text Summarisation
- Question Answering

EXAMPLES

- T5
- BART

Computational Memory Challenges

1 Parameter = 4 bytes (32 bit float)
1 Billion Parameters = 4 x 10E9 bytes = 4GB

Model Parameters = 4 bytes per parameter
2 Adam Optimisers = +8 bytes per parameter
Gradients = +4 bytes per parameter
Activations = +8 bytes per parameter

Total = 4 bytes per parameter +
20 extra bytes per parameter

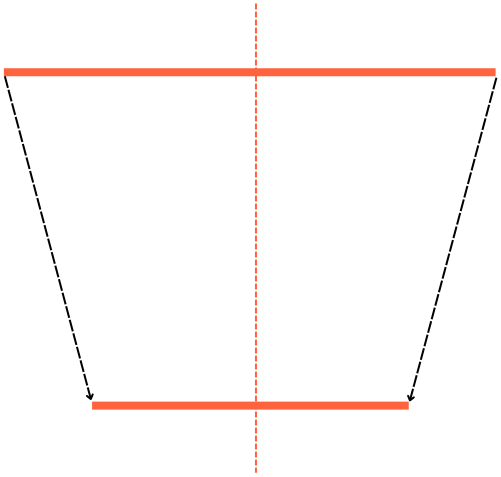
To Store

4 GB@32 bit full precision

To Train

80 GB@32 bit full precision

Optimising Memory Requirement through Quantisation



FP32
32 Bit Floating Point

Range
3e-38 to 3e+38

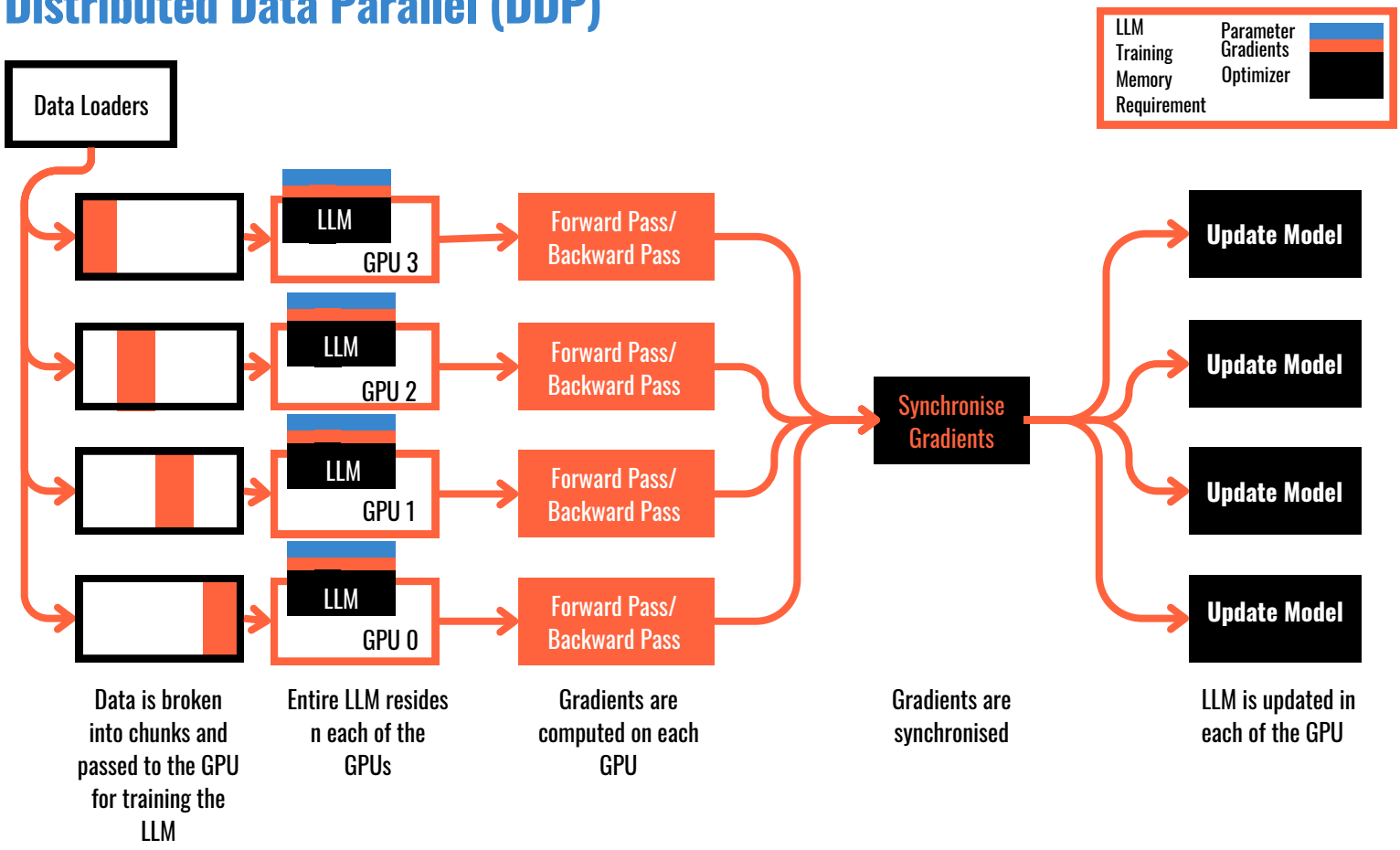
FP16 | BFLOAT16 | INT8
16 Bit Floating Point
8 Bit Integer

- Reduces the memory required to train and store models
- Quantisation projects the 32bit numbers into a lower precision space
- Quantisation aware training (QAT) learns quantisation scaling factors during training
- BFLOAT16 is a popular choice

Abhinav Kimothi

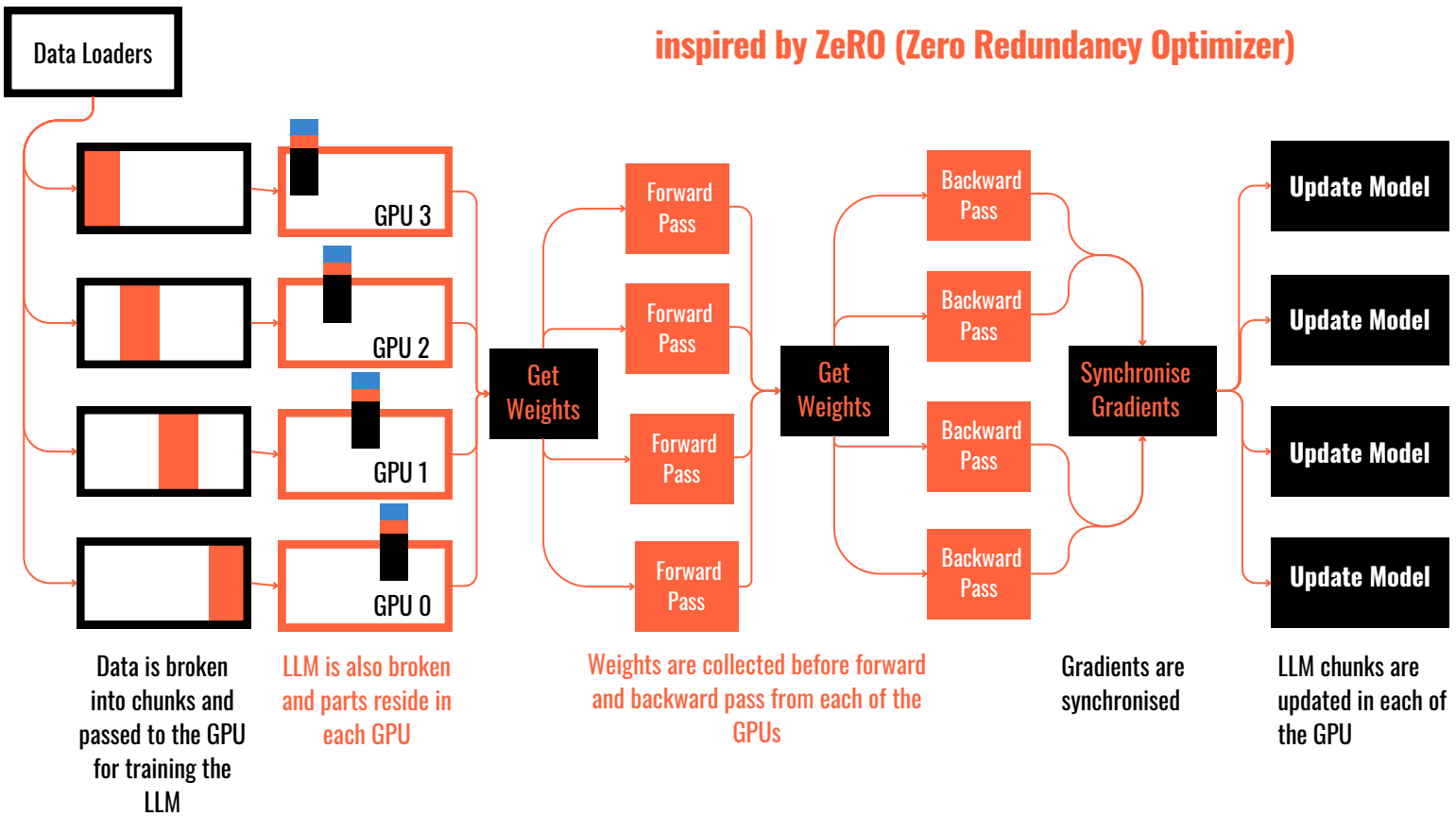
Optimising Memory Requirement through Multi-GPU Compute

Distributed Data Parallel (DDP)



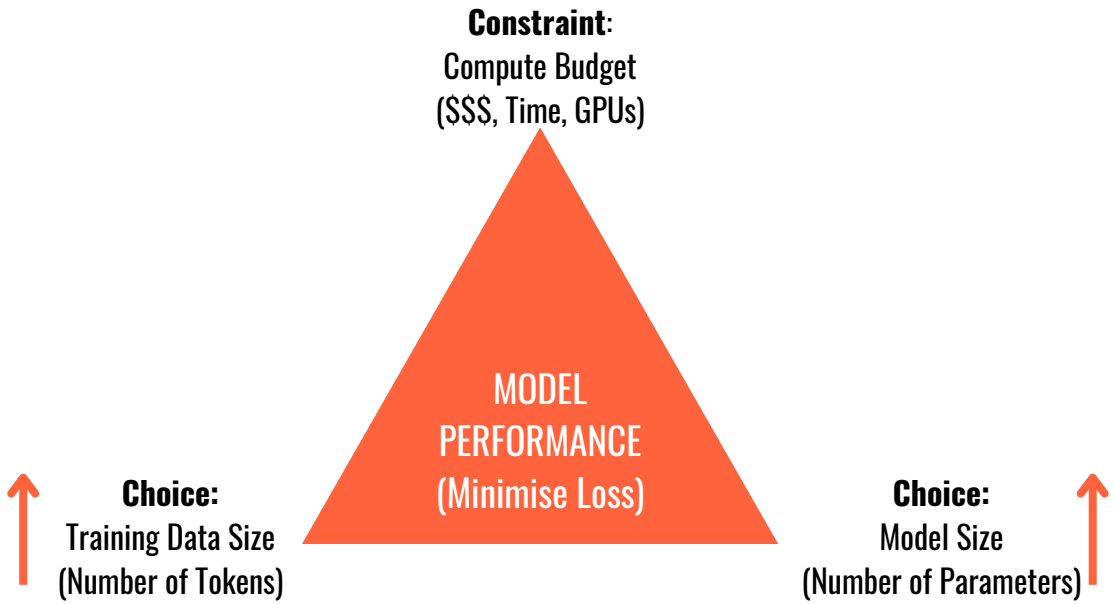
Fully Sharded Data Parallel (FSDP)

inspired by ZeRO (Zero Redundancy Optimizer)



What is the optimal computation configuration for pre-training an LLM?

- Our aim should be to minimise the loss function of the LLM.
- This can be done by providing more training tokens and/or increasing the number of parameters
- Compute cost (budget, time, GPUs) is the constraint that we operate in.



One metric to measure the cost of computation is "petaFLOP/s-day"
Number of FLOating Point operations performed at the rate of 1 petaFLOP per second for one day

1 petaFLOP per second = 10E15 or 1 Quadrillion floating point operations per second	1 petaFLOP per second - day requires 8 NVIDIA V100 chips running at full efficiency for 24 hours	1 petaFLOP per second - day requires 2 NVIDIA A100 chips running at full efficiency for 24 hours

To put this in context, OpenAIs GPT3 is a 175 B parameters model and required 3700 petaflops/s-day

Chinchilla Paper

In 2022, the paper "Training Compute-Optimal Large Language Model" studied a large number of models with different training data and model sizes to find the optimal number of tokens and parameters for a given compute budget. The authors called this optimal model "Chinchilla"

One important take-away was that the compute optimal number of training tokens should be 20 times the number of parameters

This means that smaller models can achieve the same performance as the larger ones, if they are trained on larger datasets

GENERATIVE AI WITH LARGE LANGUAGE MODELS



ABHINAV KIMOTHI

DeepLearning.AI amazon web services coursera

PART 1
LLM Pre-Training

- What is an LLM?
- Use Cases for application of LLMs
- What are Transformers? How was text generation done before Transformers?
- How does a Transformer generate Text?
- What is a Prompt?
- Generative AI Project Life Cycle.
- How do you pre-train Large Language Models?
- Challenges with pre-training LLMs.
- What is the optimal configuration for pre-training LLMs?
- When is pre-training useful?

PART 2
LLM Fine Tuning

- What is Instruction Fine Tuning?
- What Catastrophic Forgetting?
- How to Evaluate a Fine Tuned model?
- What is Parameter Efficient Fine Tuning?

PART 3
RLHF & Application

- Aligning with Human Values
- How does RLHF work?
- How to avoid Reward Hacking?
- Scaling Human Feedback : Self Supervision with Constitutional AI
- How to optimise and deploy LLMs for inferencing?
- Using LLMs in Applications
- LLM Application Architecture
- Responsible AI
- Generative AI Project Lifecycle Cheatsheet

DOWNLOAD EBOOK

(link in description)